

Discord RAG Bot – Technical Analysis Report

(Part 1)

1. PROJECT OVERVIEW

- **What:** A Discord bot that uses Retrieval-Augmented Generation (RAG) to answer community support questions by learning from historical server conversations.
- **Purpose:** Automatically reads and indexes messages from designated help/support channels, generates vector embeddings, and uses semantic search + Gemini LLM to answer user questions with context from past conversations.
- **Main Users:** Discord server administrators (setup/config) and server members (asking questions by @mentioning the bot).
- **Core Features:**
 - Slash-command-based setup for help channels and trusted roles
 - Incremental message syncing from Discord channels into a Supabase vector database
 - Semantic vector search via `pgvector` and the `match_documents` RPC function
 - AI-powered Q&A using Google Gemini (`gemini-3.1-flash-lite` for chat, `gemini-embedding-001` for embeddings)
 - Daily automated cron-based sync of all configured guilds
 - Data cleanup on bot removal from a server
 - Health-check HTTP server for deployment platforms

2. BACKEND ANALYSIS

Note: This is a backend-only project. There is no frontend.

2.1 Technology Stack

Technology	Purpose
Node.js (>=18)	Runtime environment

Technology	Purpose
discord.js v14	Discord bot framework – slash commands, events, message handling
@supabase/supabase-js v2	Supabase client for PostgreSQL + pgvector database access
@google/generative-ai v0.24	Google Gemini SDK – embeddings(<code>gemini-embedding-001</code>)and chat(<code>gemini-3.1-flash-lite</code>)
dotenv v17	Load environment variables from <code>.env</code>
node-cron v4	Schedule daily incremental sync job
http (built-in)	Minimal health-check server for deployment platforms
PM2 (via ecosystem.config.js)	Production process management with auto-restart
PostgreSQL + pgvector (via Supabase)	Vector database for storing and searching message embeddings

2.2 Folder Structure

```
discord-rag-bot/
├── index.js           # Main entry point – bot client, event handlers,
slash commands, Q&A logic
├── syncService.js     # Core sync engine – fetch messages, generate
embeddings, save to DB
├── sync.js           # Development-only manual sync script (fully
commented out)
├── schema.sql        # Full Supabase/PostgreSQL schema (tables,
indexes, RPC function)
├── ecosystem.config.js # PM2 production process manager configuration
├── package.json      # Dependencies and npm scripts
├── package-lock.json # Locked dependency tree
├── .env              # Environment variables (secrets – gitignored)
├── .env.example      # Template for required environment variables
├── .gitignore        # Ignores node_modules/, .env, dist/, build/
└── node_modules/     # Installed dependencies
```

Key files explained:

File	Role
------	------

File	Role
<code>index.js</code>	Bot startup, Discord event listeners (<code>clientReady</code> , <code>guildCreate</code> , <code>guildDelete</code> , <code>interactionCreate</code> , <code>messageCreate</code>), slash command registration, Q&A pipeline, cron scheduling, health-check server
<code>syncService.js</code>	Exports <code>syncGuildChannel</code> , <code>syncConfiguredGuild</code> , <code>fetchMessagesSince</code> , <code>saveMessageEmbeddings</code> — handles paginated message fetching from Discord, trusted-role filtering, Q&A pair construction, embedding generation, and Supabase upserts
<code>sync.js</code>	Entirely commented-out development script for manual one-off syncs
<code>schema.sql</code>	Database DDL: 3 tables (<code>guild_settings</code> , <code>guild_channels</code> , <code>discord_logs</code>), indexes, the <code>match_documents</code> vector search function, and RLS disable statements
<code>ecosystem.config.js</code>	PM2 config: app name <code>discord-rag-bot</code> , runs <code>index.js</code> , 5s restart delay, max 10 restarts

2.3 Database Analysis

Database: **Supabase PostgreSQL** with the **pgvector** extension enabled.

Table: `guild_settings`

Field	Type	Required	Description
<code>guild_id</code>	<code>text</code>	PK	Discord guild/server ID
<code>trusted_role_id</code>	<code>text</code>	No	ID of the role whose members' replies are stored as answers
<code>is_active</code>	<code>boolean</code>	Yes (default <code>true</code>)	Whether the bot is active for this guild
<code>created_at</code>	<code>timestampz</code>	Yes (default <code>now()</code>)	Row creation timestamp
<code>updated_at</code>	<code>timestampz</code>	Yes (default <code>now()</code>)	Last update timestamp

Table: `guild_channels`

Field	Type	Required	Description
-------	------	----------	-------------

Field	Type	Required	Description
id	bigint (identity)	PK	Auto-generated row ID
guild_id	text	Yes (FK → guild_settings)	Discord guild ID
channel_id	text	Yes	Discord channel ID
last_synced_message_id	text	No	ID of the last message synced for incremental sync
last_synced_at	timestampz	No	Timestamp of last successful sync
created_at	timestampz	Yes (default now())	Row creation timestamp

- **Unique constraint:** `(guild_id, channel_id)` – one row per channel per guild
- **Foreign key:** `guild_id` references `guild_settings(guild_id)` with `ON DELETE CASCADE`
- **Index:** `guild_channels_guild_id_idx` on `guild_id`

Table: discord_logs

Field	Type	Required	Description
id	text	PK	Discord message ID (used as primary key)
content	text	Yes	The Q&A pair text (<code>Q: ... A: ...</code>)
guild_id	text	Yes	Discord guild ID for scoping searches
channel_id	text	Yes	Discord channel ID
author_id	text	No	Discord user ID of the answerer
author_username	text	No	Username of the answerer
message_created_at	timestampz	No	Original Discord message timestamp
embedding	vector(3072)	Yes	3072-dimensional vector embedding from Gemini
created_at	timestampz	Yes (default now())	Row creation timestamp

- **Indexes:** `discord_logs_guild_id_idx`, `discord_logs_channel_id_idx`, `discord_logs_message_created_at_idx`

RPC Function: match_documents

`match_documents(query_embedding vector(3072), match_threshold float, match_count int)`

- Returns: `id`, `content`, `guild_id`, `channel_id`, `similarity`
- Uses **cosine distance** operator (`<=>`) to compute `1 - distance` as similarity
- Filters by `target_guild_id` and `match_threshold`
- Orders by cosine distance ascending, limits to `match_count`

Relationships

<code>guild_settings</code>	(1)	——	(N)	<code>guild_channels</code>	[CASCADE DELETE]
<code>guild_settings</code>	(1)	——	(N)	<code>discord_logs</code>	[logical, via <code>guild_id</code>]
<code>guild_channels</code>	(1)	——	(N)	<code>discord_logs</code>	[logical, via <code>channel_id</code>]

2.4 Complete API / Command Analysis

This project has **no REST API endpoints**. All interaction happens through **Discord slash commands** and **bot mentions**.

Slash Commands (registered in `registerCommands()` in `index.js`)

Command	Permission Required	Parameters	Purpose
<code>/setup-help-channel</code>	ManageGuild	<code>channel</code> (GuildText, required)	Register a text channel as a help source; triggers initial sync
<code>/remove-help-channel</code>	ManageGuild	<code>channel</code> (GuildText, required)	Remove a channel and delete all its stored data
<code>/setup-trusted-role</code>	ManageGuild	<code>role</code> (Role, required)	Set the role whose members' replies are saved as answers

Bot Mention (Q&A)

Trigger	Handler	Purpose
@mention the bot with a question	<code>messageCreate</code> event in <code>index.js</code>	Generates embedding for the question, searches <code>discord_logs</code> via <code>match_documents</code> RPC, constructs a prompt with context, calls Gemini LLM, replies with the AI answer

Health-Check Endpoint

Method	Route	Purpose
GET (any)	<code>http://localhost:{PORT}</code>	Returns <code>200 OK</code> — used by deployment platforms for uptime monitoring

2.5 Middleware Analysis

This project does **not use Express** or any HTTP middleware framework. It is a Discord bot built on `discord.js` events. However, there are equivalent access-control checks:

Permission Check (in `interactionCreate` handler, `index.js` line 239)

```
if (!interaction.memberPermissions?.has(PermissionFlagsBits.ManageGuild))
```

- **What:** Checks if the user invoking a slash command has the `ManageGuild` Discord permission
- **When:** Before any slash command logic executes
- **Effect:** Replies with ephemeral "Only server managers can run this command." if unauthorized

Bot/Empty Message Filter (in `messageCreate` handler, `index.js` lines 367-369)

```
if (!message.guild || message.author.bot) return;  
if (!message.content) return;  
if (!message.mentions.has(client.user)) return;
```

- Ignores DMs, bot messages, empty messages, and messages that don't @mention the bot

Channel Type Validation (`index.js` line 308)

- Validates that the chosen channel is a text-based channel with readable messages

Retry Middleware (`syncService.js` — `withRetry()`)

- Wraps embedding API calls with exponential backoff: 3 retries, delay = `2000ms * attempt`

Execution Order for a Slash Command

Discord Event → `interactionCreate` listener

- `isChatInputCommand()` check
 - `ManageGuild` permission check
 - Command-specific logic (`setup-help-channel` / `remove-help-channel` / `setup-trusted-role`)
-

2.6 Authentication Flow

Bot Authentication

- The bot authenticates to Discord using `client.login(process.env.DISCORD_TOKEN)` — a long-lived bot token
- Supabase is accessed via `createClient(SUPABASE_URL, SUPABASE_KEY)` using the anon/public key
- Gemini is accessed via `new GoogleGenerativeAI(GEMINI_API_KEY)`

User Authorization

- **No user login/signup** — this is a Discord bot, not a web app
- Authorization is handled by Discord's permission system: only users with `ManageGuild` permission can run admin slash commands
- The `trusted_role_id` mechanism provides content-level authorization: only messages from trusted-role members or the server owner are indexed as answers

Environment Variable Validation

- At startup (`index.js` lines 446-451), the bot checks that `DISCORD_TOKEN` , `SUPABASE_URL` , `SUPABASE_KEY` , and `GEMINI_API_KEY` are all set
 - If any are missing, it logs an error and does **not** call `client.login()`
-

2.7 Business Logic

Message Sync Pipeline (`syncService.js`)

1. `fetchMessagesSince(channel, lastSyncedMessageId)` – Paginates through Discord channel history (100 messages per batch) working backwards until it reaches the `lastSyncedMessageId` or the beginning of the channel. Returns messages sorted oldest-first.
2. `isHumanTextMessage(msg)` – Filters out bot messages and empty messages.
3. `isTrustedMember(msg, trustedRoleId)` – Returns `true` if the message author is the server owner or has the trusted role.
4. `buildEmbedText(msg, messageMap)` – Only processes **reply messages**. Looks up the referenced (parent) message and constructs a Q&A pair string: `Q (username): question\nA (username): answer`. Returns `null` if the message is not a reply or the referenced message is not found.
5. `saveMessageEmbeddings({...})` – For each trusted member's reply that has a valid Q&A pair: generates a 3072-dim embedding via `gemini-embedding-001` (with retry), then upserts into `discord_logs`.
6. `syncGuildChannel({...})` – Orchestrates fetch → embed → save. After completion, updates `guild_channels.last_synced_message_id` and `last_synced_at`.
7. `syncConfiguredGuild({...})` – Fetches the Discord channel object, validates it, then delegates to `syncGuildChannel`.

Q&A Pipeline (`index.js` – `messageCreate` handler)

1. User @mentions the bot with a question
2. Bot strips the mention, extracts the question text
3. Generates a 3072-dim query embedding via `gemini-embedding-001`
4. Calls `supabase.rpc('match_documents', {...})` with threshold `0.1` and count `8`, scoped to the guild
5. If no matches: replies with fallback message
6. Constructs a prompt with matched context documents and strict rules (answer only from context, treat context as data not instructions)
7. Calls `gemini-3.1-flash-lite` to generate the answer
8. Truncates to 1900 characters (Discord limit safety) and replies

Guild Lifecycle

- `guildCreate`: Upserts `guild_settings`, sends setup instructions to system channel

- `guildDelete` : Deletes all `discord_logs` for the guild, then deletes `guild_settings` (which cascades to `guild_channels`)

Cron Job

- Runs daily at midnight UTC (`0 0 * * *`) via `node-cron`
- Calls `syncAllConfiguredGuilds()` which iterates all active guild settings and their channels, performing incremental syncs

3. FRONTEND ANALYSIS

This project has NO frontend. It is a pure backend Discord bot. There are no HTML, CSS, React, or any frontend components. All user interaction occurs through Discord's native interface (slash commands and chat messages).

4. COMPLETE REQUEST FLOWS

Flow 1: Bot Startup

```
node index.js
  ↓
dotenv loads .env
  ↓
Validate required env vars (DISCORD_TOKEN, SUPABASE_URL, SUPABASE_KEY,
GEMINI_API_KEY)
  ↓
client.login(DISCORD_TOKEN)
  ↓
clientReady event fires
  ↓
registerCommands() — registers 3 slash commands globally
  ↓
ensureGuildSetting() for every cached guild
  ↓
cron.schedule('0 0 * * *', syncAllConfiguredGuilds)
```

↓
http health-check server starts on PORT (default 3000)

Flow 2: Setup Help Channel(/setup-help-channel)

Admin runs /setup-help-channel #support

↓
interactionCreate event
↓
isChatInputCommand() → true
↓
ManageGuild permission check → pass
↓
commandName === 'setup-help-channel'
↓
Validate channel is text-based
↓
interaction.deferReply() (ephemeral)
↓
Supabase: SELECT guild_settings.trusted_role_id WHERE guild_id
↓
Supabase: SELECT guild_channels WHERE guild_id + channel_id (check existing)
↓
Supabase: UPSERT guild_channels (guild_id, channel_id)
↓
interaction.editReply() – confirms setup
↓
runInitialSync() fires asynchronously
↓
syncGuildChannel() → fetchMessagesSince() → saveMessageEmbeddings()
↓
For each trusted reply: Gemini embedContent() → Supabase UPSERT discord_logs
↓
Update guild_channels.last_synced_message_id
↓
interaction.followUp() – reports sync results

Flow 3: User Asks a Question (@mention)

User sends: "@Bot how do I reset my password?"

↓

messageCreate event

↓

Filter: not DM, not bot, has content, mentions bot → pass

↓

Supabase: SELECT guild_settings + guild_channels WHERE guild_id

↓

Validate bot is configured (has channels)

↓

Strip mention, extract question text

↓

Gemini: embeddingModel.embedContent(question) → 3072-dim vector

↓

Supabase: RPC match_documents(vector, threshold=0.1, count=8, guild_id)

↓

If no matches → reply with FALLBACK_ANSWER

↓

Build context string from matched docs (with similarity scores)

↓

Construct prompt with context + rules + question

↓

Gemini: chatModel.generateContent(prompt)

↓

truncateForDiscord(answer, 1900)

↓

message.reply(answer)

Flow 4: Remove Help Channel (/remove-help-channel)

Admin runs /remove-help-channel #support

↓

interactionCreate → permission check → pass

↓

interaction.deferReply() (ephemeral)

↓

Supabase: DELETE FROM discord_logs WHERE guild_id AND channel_id

↓

Supabase: DELETE FROM guild_channels WHERE guild_id AND channel_id

↓

interaction.editReply() – confirms removal

Flow 5: Setup Trusted Role (/setup-trusted-role)

Admin runs /setup-trusted-role @Support

↓

interactionCreate → permission check → pass

↓

Supabase: UPSERT guild_settings SET trusted_role_id = role.id

↓

interaction.reply() – confirms role set (ephemeral)

Flow 6: Bot Removed From Server

Bot is kicked/removed from guild

↓

guildDelete event

↓

Supabase: DELETE FROM discord_logs WHERE guild_id

↓

Supabase: DELETE FROM guild_settings WHERE guild_id

↓

(guild_channels rows cascade-deleted via FK)

Flow 7: Daily Cron Sync

Cron fires at 00:00 UTC daily

↓

syncAllConfiguredGuilds()

↓

Supabase: SELECT guild_settings WHERE is_active + JOIN guild_channels

↓

For each guild → for each channel:

↓

syncConfiguredGuild() → client.channels.fetch()

↓

syncGuildChannel() → fetchMessagesSince(lastSyncedMessageId)

↓

saveMessageEmbeddings() → embed + upsert for each trusted reply

↓

Update guild_channels.last_synced_message_id

5. DATA FLOW

Question → Answer Data Flow

```
User Message (Discord)
  ↓ [discord.js parses Message object]
Plain text question (string)
  ↓ [Gemini embedContent()]
3072-dim float vector (array)
  ↓ [Supabase RPC match_documents - cosine similarity]
Matched documents (JSON array: {id, content, guild_id, channel_id,
similarity})
  ↓ [String concatenation into context]
Prompt string (context + rules + question)
  ↓ [Gemini generateContent()]
AI answer (string)
  ↓ [truncateForDiscord - max 1900 chars]
Discord reply (string)
```

Sync Data Flow

```
Discord Channel History
  ↓ [channel.messages.fetch() - paginated, 100/batch]
Array of Discord Message objects
  ↓ [Filter: isHumanTextMessage + isTrustedMember + buildEmbedText]
Q&A pair strings ("Q (user): ...\nA (user): ...")
  ↓ [Gemini embeddingModel.embedContent()]
3072-dim vector per Q&A pair
  ↓ [Supabase upsert - JSON payload]
discord_logs row {id, content, guild_id, channel_id, author_id,
author_username, message_created_at, embedding}
```

Serialization

- **Discord → Bot:** discord.js deserializes WebSocket payloads into JS objects
 - **Bot → Supabase:** Supabase client serializes JS objects to JSON for REST API calls
 - **Bot → Gemini:** Google AI SDK serializes requests to JSON for gRPC/REST
 - **Supabase → Bot:** JSON responses deserialized to JS objects by Supabase client
-

6. DATABASE OPERATIONS

Insert / Upsert

- **guild_settings upsert** – `ensureGuildSetting()`, `/setup-trusted-role`, `/setup-help-channel` – uses `onConflict: 'guild_id'`
- **guild_channels upsert** – `/setup-help-channel` – uses `onConflict: 'guild_id,channel_id'` with `ignoreDuplicates: true`
- **discord_logs upsert** – `saveMessageEmbeddings()` – uses message ID as primary key for idempotent inserts

Read

- **guild_settings select** – `messageCreate` handler fetches settings + joined `guild_channels` for the guild
- **guild_settings select** – `syncAllConfiguredGuilds()` fetches all active guilds with their channels
- **guild_channels select** – `/setup-help-channel` checks for existing channel record
- **match_documents RPC** – Vector similarity search on `discord_logs` filtered by `guild_id`

Update

- **guild_channels update** – After sync, updates `last_synced_message_id` and `last_synced_at`

Delete

- **discord_logs delete** – `/remove-help-channel` deletes logs for a specific channel; `guildDelete` deletes all logs for the guild
- **guild_channels delete** – `/remove-help-channel` deletes the channel row
- **guild_settings delete** – `guildDelete` deletes settings (cascades to `guild_channels`)

Indexes

Index	Table	Column(s)
-------	-------	-----------

Index	Table	Column(s)
<code>guild_channels_guild_id_idx</code>	<code>guild_channels</code>	<code>guild_id</code>
<code>discord_logs_guild_id_idx</code>	<code>discord_logs</code>	<code>guild_id</code>
<code>discord_logs_channel_id_idx</code>	<code>discord_logs</code>	<code>channel_id</code>
<code>discord_logs_message_created_at_idx</code>	<code>discord_logs</code>	<code>message_created_at</code>

Aggregation / Joins

- **Supabase relational query:** `guild_settings` → `guild_channels` via
`select('guild_id, trusted_role_id, guild_channels(channel_id, last_synced_message_id)')` (PostgREST embedded resource)
- No explicit SQL JOINS – the RPC function queries only `discord_logs`

Transactions

- No explicit transactions are used. Operations rely on individual upserts/deletes.